

Requirement Analysis Phase Solution Requirements

Date	01 October 2026
Team ID	SWTID-2026-7059
Project Name	FitBuddy
Maximum Marks	4 Marks

Problem Statement:

Many individuals struggle to create effective, personalized fitness plans due to lack of domain expertise, time constraints, and high cost of personal trainers. Generic fitness programs fail to account for individual differences in age, body weight, fitness goals, and workout intensity preferences. **FitBuddy** solves this by using Google Gemini AI to generate tailored 7-day workout plans and nutrition tips — instantly and at no cost to the user.

Objective:

- To develop a web application that generates AI-powered personalized 7-day workout plans
- To provide goal-based nutrition and recovery tips using Gemini Flash model
- To allow users to refine their plans using natural language feedback
- To persist user profiles and workout plans in a database for reference
- To provide an admin dashboard to view and manage all user records

Functional Requirements:

The system shall satisfy the following functional requirements:

Req. ID	Requirement	Description	Priority
FR-01	User Profile Input	The system shall accept and validate: Name (max 100 chars), User ID (positive integer), Age (10–100), Weight (20–400 kg), Fitness Goal (free text), and Workout Intensity (Low / Medium / High).	High
FR-02	AI Workout Plan Generation	The system shall call Google Gemini Pro API with the user's profile to generate a structured 7-day workout plan. Each day includes warm-up, main workout (exercises,	High

Req. ID	Requirement	Description	Priority
		sets & reps), and cooldown. Minimum one rest or active recovery day must be included.	
FR-03	AI Nutrition Tip Generation	The system shall generate a personalized nutrition or recovery tip using Gemini Flash based on the user's fitness goal. A pre-defined fallback tip shall be served if the AI is unavailable.	Medium
FR-04	Plan Persistence	The system shall save user profiles and generated workout plans in a SQLite database using SQLAlchemy ORM. Users and plans are linked by unique User ID. Regenerating a plan replaces the original and clears any updated plan.	High
FR-05	Feedback-Based Plan Update	Users shall submit textual feedback (3–1000 chars) on their current plan. The system shall send the original plan and feedback to Gemini Pro to generate a revised full 7-day plan stored as the updated plan.	High
FR-06	Admin Dashboard	An admin view (/view-all-users) shall list all registered users with original and updated plans. Admins shall delete any user and their plans via a delete action.	Medium
FR-07	REST JSON API	The system shall expose RESTful JSON endpoints for programmatic access: generate workout, generate nutrition tip, save plan, update plan via feedback, and list all users and plans.	Low

Non-Functional Requirements:

Req. ID	Category	Requirement	Target / Metric
NFR-01	Performance	Plan generation response time	≤ 60 seconds (Gemini API dependent); loading state shown in UI
NFR-02	Reliability	Gemini model fallback chain	Tries ≥ 3 model alternatives before failing; fallback nutrition tips always available offline

Req. ID	Category	Requirement	Target / Metric
NFR-03	Security	API key protection	GOOGLE_API_KEY stored in .env, never exposed in HTML or source code; .gitignore excludes .env
NFR-04	Scalability	Serverless deployment	Deployed on Vercel Python serverless; stateless per-request; DATABASE_URL configurable for cloud DB
NFR-05	Usability	Responsive UI	Works on mobile ($\geq 320\text{px}$) and desktop; inline error messages; loading state feedback
NFR-06	Maintainability	Modular code structure	Separate modules: routes.py, database.py, gemini_generator.py, gemini_flash_generator.py, schemas.py, nutrition.py
NFR-07	Validation	Input sanitization	All user inputs validated via Pydantic v2 schemas; errors shown inline with field-level messages

Constraints & Assumptions:

S.No	Type	Description
1	API Dependency	Workout plan quality and availability depend on Google Gemini API quota and service availability.
2	Database	SQLite is used for simplicity; not suitable for high-concurrency multi-user production loads.
3	Authentication	No user authentication system — User ID is self-supplied; admin dashboard is publicly accessible.
4	Deployment	On Vercel, SQLite DB is stored in /tmp which is ephemeral (data may reset on cold starts).
5	Language	Application is English-only; no multi-language support currently implemented.

References:

1. [Google Gemini API Documentation – https://ai.google.dev/gemini-api/docs](https://ai.google.dev/gemini-api/docs)
2. [FastAPI Official Documentation – https://fastapi.tiangolo.com/](https://fastapi.tiangolo.com/)
3. [Pydantic v2 Field Validation – https://docs.pydantic.dev/latest/concepts/validators/](https://docs.pydantic.dev/latest/concepts/validators/)

4. [SQLAlchemy ORM – https://docs.sqlalchemy.org/en/20/](https://docs.sqlalchemy.org/en/20/)